

Machine Learning - II

Assignment - II (ConvNets for CV)

April 28, 2026

NOTE: If you can solve this assignment, you can confidently say that you have a good understanding and applied capacity for ConvNets for Production Line, Transfer Learning and Knowledge Distillation

Client constraint

Your client is adding a camera to a production line. Every frame must be classified fast enough to support a real-time decision system. Accuracy matters, but the deployed model must also be small. Your final submitted model must be a lightweight student model with **strictly fewer than 500,000 parameters**.

1 Assignment Overview

In this assignment, you will act as the machine learning engineer for a client-facing computer vision system. The client has provided a labeled training set and a larger unlabeled image set. Your job is to produce a compact classification model that can run in a production-line camera system while preserving as much accuracy as possible.

You will also train a large teacher model, including a pretrained model, to help learn from the labeled and unlabeled data. However, the model delivered to the client must be the small student model only. The teacher is a training tool, not a deployment artifact.

Main objective

Train and submit a PyTorch image classifier with **strictly fewer than 500,000 total parameters**. Your report must explain how you designed the model and trained it.

Learning goals

- Build deep convolutional networks using PyTorch modules such as `nn.Conv2d`, pooling, activations, and normalization layers.
- Use training-specific tools such as BatchNorm, LayerNorm, Dropout, optimizers, learning-rate schedules, and data augmentation.
- Fine-tune a pretrained teacher model to obtain a strong reference model for the task.
- Use unlabeled data through teacher-student knowledge distillation.
- Compare models using validation accuracy, parameter count, reproducibility, and deployment constraints.

2 Data and Starter Kit

You will receive a `starter-kit.zip` file and a dataset directory.

Path or file	Purpose
<code>/train</code>	Labeled training images. Use these images to train, validate, and debug your classifier.
<code>/unlabeled</code>	Unlabeled images from the same deployment setting. You may use them for teacher prediction, pseudo-labeling, and distillation.
<code>train_baby.py</code>	Example training script for a small student model. This is just a sample that may be substantially suboptimal. You will modify it per your design.
<code>train_teacher.py</code>	Example script for fine-tuning a larger teacher model. You may use any pretrained teacher you like.
<code>distill.py</code>	Example script for teacher-student distillation using labeled and/or unlabeled data.

3 Deployment Rules and Technical Constraints

Strict model-size rule

Your submitted student model must have **strictly fewer than 500,000 total parameters**. All parameters stored in the submitted model count, including frozen parameters, BatchNorm parameters, embeddings, classifier layers, and any other learned tensors.

Student ID

You will submit with your 8-digit student ID and the Leaderboard will populate your name.

Allowed	Not allowed
Any PyTorch architecture under the parameter limit, including Conv2d blocks, residual blocks, depthwise or grouped convolutions, global average pooling, and small MLP heads.	Submitting a teacher model, ensemble, pretrained backbone, or any model with 500,000 or more total parameters.
BatchNorm, LayerNorm, Dropout, weight decay, data augmentation, learning-rate schedules, early stopping, and any optimizer from <code>torch.optim</code> such as SGD, Adam, AdamW, Adamax, or RMSprop.	Changing the evaluation protocol, using hidden test labels, hard-coding predictions, or relying on manual inspection of test images.
A pretrained teacher model for training only. Examples: ResNet18, ResNet50, EfficientNet variants, ConvNeXt variants, or other image classifiers available in standard libraries.	Shipping the pretrained teacher checkpoint to the client or depending on the teacher at inference time.
Unlabeled data from <code>/unlabeled</code> for teacher prediction, pseudo-labels, consistency training, or distillation.	Using outside labeled data unless the instructor explicitly approves it.
Engineering tip	
Small models can overfit quickly. Start with a simple ConvNet, verify it learns on a tiny subset, then add capacity, normalization, augmentation, and regularization one change at a time.	

4 Knowledge Distillation

Knowledge distillation trains a compact student model to imitate a stronger teacher. Instead of learning only from hard labels, the student also learns from the teacher's probability distribution over classes. This distribution can contain useful information about class similarity and uncertainty.

The temperature parameter T controls how soft the teacher distribution is:

```
teacher_probs_T = softmax(teacher_logits / T)
student_log_probs_T = log_softmax(student_logits / T)
```

Temperature	What it usually does	What to look for
$T = 1$	Teacher outputs are not softened. This is closest to ordinary probability matching.	May work well if the teacher is calibrated, but the targets can be very sharp.
$T = 2$ or 4	Class probabilities become softer, exposing more information about non-winning classes.	Often a useful range for comparing accuracy, calibration, and student stability.
$T = 8$ or higher	The distribution becomes much flatter. This may reveal broad similarities, but can also reduce useful signal.	Check whether validation accuracy improves or whether training becomes too weak or noisy.

A common distillation objective combines the ordinary supervised loss and a teacher-student loss:

```
loss = (1 - alpha) * cross_entropy(student_logits, labels)
loss += alpha * (T * T) * kl_divergence(
    log_softmax(student_logits / T),
    softmax(teacher_logits / T)
)
```

Required T analysis

Your report must compare multiple values of T and explain what changed. Include both numbers and interpretation: Did higher T help? Did it make the student less overconfident? Did it hurt because the teacher distribution became too flat?

Final reminder

Trying to find the source of the dataset on web, and utilizing other labeled data that was not delivered to you with the assignment will be considered cheating.

Submission Link

Submit your model weights to this link: <https://ml2-hw2.onrender.com/>